



PDF Download
3594805.3607131.pdf
11 February 2026
Total Citations: 2
Total Downloads: 750

Latest updates: <https://dl.acm.org/doi/10.1145/3594805.3607131>

RESEARCH-ARTICLE

General Boolean Function Benchmark Suite

ROMAN KALKREUTH, University of Paris, Paris, Ile-de-France, France

ZDENĚK VAŠIČEK, Brno University of Technology, Brno, South Moravian Region, Czech Republic

JAKUB HUSA, Brno University of Technology, Brno, South Moravian Region, Czech Republic

DIEDERICK VERMETTEN, Leiden University, Leiden, Zuid-Holland, Netherlands

FURONG YE, Leiden University, Leiden, Zuid-Holland, Netherlands

THOMAS H W BÄCK, Leiden University, Leiden, Zuid-Holland, Netherlands

Open Access Support provided by:

Brno University of Technology

University of Paris

Leiden University

Published: 30 August 2023

[Citation in BibTeX format](#)

FOGA '23: Foundations of Genetic Algorithms XVII

August 30 - September 1, 2023
Potsdam, Germany

Conference Sponsors:
SIGEVO

General Boolean Function Benchmark Suite

Roman Kalkreuth
r.t.kalkreuth@liacs.leidenuniv.nl
Computer Lab of Paris 6, Sorbonne
University
Paris, France

Zdeněk Vašíček
vasicek@fit.vut.cz
Brno University of Technology
Brno, Czech Republic

Jakub Husa
ihusa@fit.vut.cz
Brno University of Technology
Brno, Czech Republic

Diederick Vermetten
d.l.vermetten@liacs.leidenuniv.nl
Leiden Institute of Advanced
Computer Science, Leiden University
Leiden, Netherlands

Furong Ye
f.ye@liacs.leidenuniv.nl
Leiden Institute of Advanced
Computer Science, Leiden University
Leiden, Netherlands

Thomas Bäck
T.H.W.Baek@liacs.leidenuniv.nl
Leiden Institute of Advanced
Computer Science, Leiden University
Leiden, Netherlands

ABSTRACT

Just over a decade ago, the first comprehensive review on the state of benchmarking in Genetic Programming (GP) analyzed the mismatch between the problems that are used to test the performance of GP systems and real-world problems. Since then, several benchmark suites in major GP problem domains have been proposed over time, filling some of the major gaps. In the framework of the first review about the state of benchmarking in GP, logic synthesis (LS) was classified as one of the major GP problem domains. However, a diverse and accessible benchmark suite for LS is still missing. In this work, we propose a benchmark suite for LS that covers different types of Boolean functions that are commonly used in the field of GP. We analyze the complexity of the proposed benchmark by using popular complexity measures that are commonly used to classify and characterize Boolean functions and digital circuits.

CCS CONCEPTS

• **Computing methodologies** → **Discrete space search; Genetic programming; Evolvable hardware**; • **Hardware** → **Combinational synthesis**.

KEYWORDS

Benchmarking, Logic Synthesis, Genetic Programming

ACM Reference Format:

Roman Kalkreuth, Zdeněk Vašíček, Jakub Husa, Diederick Vermetten, Furong Ye, and Thomas Bäck. 2023. General Boolean Function Benchmark Suite. In *Proceedings of the 17th ACM/SIGEVO Conference on Foundations of Genetic Algorithms (FOGA '23)*, August 30-September 1, 2023, Potsdam, Germany. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3594805.3607131>

1 INTRODUCTION

Recent community-wide initiatives on benchmarking in Evolutionary Computation (EC)¹ have sparked new interest in this area

¹<https://sites.google.com/view/benchmarking-network/home>



This work is licensed under a Creative Commons Attribution International 4.0 License.

FOGA '23, August 30-September 1, 2023, Potsdam, Germany

© 2023 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-0202-0/23/08.

<https://doi.org/10.1145/3594805.3607131>

and have pointed out the significance of good benchmarks for the evaluation and comparison of heuristic algorithms [2]. Overall, Benchmarking has grown into a diverse and mature field in EC which also impacts Genetic Programming (GP). Just over a decade ago, McDermott et al. [39] published the first review on benchmarking in GP [8, 11, 18, 31]. The work highlighted the damaging gap between benchmarks commonly used to test GP systems and real-world problems. The effort of McDermott et al. was followed by a community survey [61] which found support for blacklisting commonly used benchmarks that have important flaws and proposed replacement problems. Since then, several benchmark suites have emerged in major GP problem domains. At last year's GECCO conference, the work of McDermott et al. [39] was awarded the SIGEVO Impact award which triggered reflection on the developments of the last decades. Very recently, a follow-up article on the state and of development of Benchmarking has been published by McDermott et al. [38]. Besides reviewing GP benchmark suites which have been proposed over the last years, the lack of a Boolean function benchmark suite for logic synthesis (LS) has been identified as one of the major gaps. Although some benchmarks for LS have been proposed, a general benchmark suite for this problem domain is still missing. Moreover, the authors state that for the future of benchmarking in LS it might be useful to further increase the diversity of benchmarks by exploring new Boolean function problems and curating these problems into a new benchmark suite. Therefore, in this work, we follow up the suggestion of McDermott et al. [38] and consider benchmarking in LS from a general perspective. We propose a benchmark suite for LS that bundles a diverse set of Boolean functions selected from major categories which were commonly used in former GP research. Our benchmark suite aims on the synthesis of Boolean functions from scratch which has been considered a challenging task for GP methods [56]. We analyze the complexity of the proposed benchmarks to provide an overview of the corresponding characteristics. We also present baseline results that have been obtained with Cartesian Genetic Programming (CGP) [42], a graph-based GP variant that has been frequently used for LS in the past.

The rest of this document is structured as follows: Section 2 reflects on the developments on benchmarking in EC and GP in specific. Section 3 describes and reviews the problem domain of logic synthesis in the field of GP. In Section 4, we propose our new benchmark

suite, analyze the complexity of the proposed problems, and describe the respective evaluation methods. Section 5 describes CGP, and presents baseline results. Section 6 addresses points that are worthy of discussion, and offers guidelines for using the benchmarks. Finally, Section 7 concludes the outcome of our benchmark suite and outlines future directions of our research.

2 BENCHMARKING IN EVOLUTIONARY COMPUTATION

In the wider field of evolutionary computation, the community is aware of the importance of benchmarking, and many benchmarking studies have been conducted. Following the recent discussion in [2], it is commonly accepted that benchmarking shall aim and help assess problems and algorithms, detect the sensitivity of algorithms' performance, explore promising performance, complement theoretical study, and address reproducibility and algorithm development. Referring to this list of goals, we witness the success achieved by the benchmarking tools, i.e., C^Omparing Continuous Optimizers (COCO) [13] and IOHprofiler [9, 59], in the past years. For example, COCO, integrated with several Black-box Optimization Benchmarking (BBOP) problem suites [54], provides a well-established platform for continuous, noisy, multi-objective, and mixed-integer optimization. The BBOP suites offer opportunities for fair comparisons of evolution strategies, differential evolution, etc., and enhance the development of these algorithms. IOHprofiler, which was released later, provides a modular-designed benchmarking platform such that the tool can easily integrate with various types of customized problems [9]. While additional customized problems, such as submodular optimization [44], are integrated into IOHprofiler, we observed that related benchmark studies based on its first problem suite of pseudo-Boolean optimization (PBO) already encouraged empirical analysis, theoretical results and even hyperparameter optimization work for evolutionary and genetic algorithms [64]. Moreover, both tools host a database of algorithms' performance for specific problem suites, which maintains reproducibility and comparison baselines for future study. The successful cases mentioned above have shown that benchmarking work is worthwhile, and we likewise desire a robust, reproducible, and fair benchmarking platform for enhancing the development of EC.

2.1 Developments in Genetic Programming

Since the publication of the original GP benchmarking papers several benchmark suites for GP have been published and benchmarking in GP has progressed then. The General Program Synthesis Benchmark Suite (PSB) as proposed by Helmuth and Spector [17] bundles 29 general program synthesis benchmark problems that have been selected from various sources of computer science programming problems on an introductory level. According to McDermott et al. [38], PSB closed one of the major gaps which has been issued in the original GP benchmarks papers. A new set of more difficult benchmark problems curated in the second program synthesis benchmark suite has been proposed recently [16]. The intention to propose a second benchmark suite for program synthesis was to set new baselines in this problem domain and to move beyond the first benchmark suite which has become outdated over time. A broad benchmarking of recent GP approaches to symbolic

regression in the context of state-of-the-art machine learning (ML) approaches has been presented by Orzechowski et al. [45]. For the benchmarking, a set of 100 regression benchmark problems was curated from open-source repositories across the web. The resulting benchmarking framework *SRBench*² now consists of 14 symbolic regression methods and 7 other ML methods and provides access to 252 datasets from the Penn Machine Learning Benchmarks (PMLB)³. The Diverse and Generative ML benchmark (DIGEN)⁴ as proposed by Orzechowski and Moore [46] is a modern ML benchmark consisting of 40 synthetic binary problems which have been designed to differentiate the performance of some of the leading ML methods. DIGEN is shipped with a package that can be used to perform reproducible benchmarking and to simplify comparison of performance among the methods. A benchmark based on problems taken from the LGSynth91 benchmark suite [63] created by the logic synthesis community has been used by Vasicek and Sekanina for the synthesis of combinational logic circuits from scratch [56]. To overcome the limitation of scalability of evaluation, the candidate solutions were transformed to a corresponding binary decision diagram (BDD) to speed up the fitness computation. A benchmark for designing combinational logic circuits with metaheuristics approaches such as CGP has been proposed by de Souza et al. [10]. The benchmark covers a set of digital logic circuits design problems that have been also selected from the LGSynth91 benchmark suite and several methodologies for performance evaluation.

3 LOGIC SYNTHESIS IN GENETIC PROGRAMMING

Logic synthesis [12, 15] (LS) in GP can be considered a black-box and optimization problem domain that has played a major role in the application scope of GP research throughout its history. In general, LS by means of GP refers to the application of GP models to synthesize expressions that match the input-output mapping of Boolean functions (Definition 3.1). Boolean functions can be formally expressed and mathematically described with Boolean expressions (Definition 3.2). LS as tackled with the GP paradigm predominantly addresses two major tasks in this problem domain:

- (1) Synthesis of a Boolean expression that produces the correct output given the inputs of the Boolean function.
- (2) Optimization of the Boolean expression that represents a certain Boolean function.

The latter task is approached by defining one or more optimization objectives. Both tasks are performed in accordance with Boolean logic and algebra. Besides, algebraic expressions, Boolean functions are commonly represented with truth tables that describe the input-output mapping of the respective function.

Definition 3.1 (Boolean Function). A Boolean function is a mathematical function where the values of the respective arguments and the result vary in a set with two elements (usually $\{0, 1\}$ or $\{\text{true}, \text{false}\}$). A Boolean function of degree n with one output is defined as $f : \{0, 1\}^n \mapsto \{0, 1\}$ and a Boolean function with multiple outputs as $f : \{0, 1\}^n \mapsto \{0, 1\}^m$ where $m > 1$.

²<https://github.com/cavalab/srbench>

³<https://github.com/EpistasisLab/pmlb>

⁴<https://github.com/EpistasisLab/digen>

Definition 3.2 (Boolean Expression). A Boolean expression is an algebraic expression of a Boolean function that can be a finite combination of Boolean constants, variables, and logical operators. The evaluation of a Boolean expression leads to a Boolean value.

3.1 Logic Synthesis of Boolean Functions with Known Input-output Mapping

As a nature-inspired search heuristic for automatic programming, GP is well suited for LS since various GP representation models can easily be applied to this task. LS as an application field for GP was popularized by Koza by representing LISP programs as parse trees [31–35]. Since various Boolean expressions can be formulated as LISP S-expressions, Koza used his approach to evolve Boolean expressions for Boolean functions like digital multiplexers and parity. Prior to tree-based GP, Learning Classifier Systems (LCS) [3, 21, 22] have been used to learn multiplexer functions [62].

Besides the parse-tree representation model of GP, graph-based representation models were proposed [27, 40, 43, 49]. One of the most well-known representation models is CGP which will be described in Section 5. Commonly used multiple-output functions for the evaluation and comparison of graph-based GP models and corresponding operators are arithmetic functions such as the digital adder and multiplier as well as combinational functions [1, 20, 29, 51]. Since a large part of Boolean functions can be implemented as digital circuits a real-world application of graph-based GP is located in the field of evolvable hardware [50]. This type of application poses the requirement on GP systems to be able to synthesize combinational circuits from scratch which has been considered to be a challenging task for graph-based GP [56].

3.2 Logic Synthesis of Black Box Problems

In some applications, the goal of LS is not to find a Boolean expression that matches the input-output mapping of a specific Boolean function but to find any Boolean function with some specific properties. These are collectively known as black box problems.

One group of problems that falls into this category, and that will be used as a general representative of black box problems in our benchmark, is the design of cryptographic Boolean functions. These functions are crucial in symmetric cryptography and can be found in both stream ciphers (where are used to mask the internal state of a pseudo-random number generator) and block ciphers (where they are used as substitution boxes to provide cryptographic confusion), and can also be used to protect the physical implementation of other applications by masking the intermediary values processed by the algorithm [24, 25, 47, 48].

To be useful in any of these scenarios, a Boolean function must possess one or more cryptographic properties, each of which provides protection against some type of cryptographic attack. Which attacks are possible and which properties are therefore desired depends on the intended use of the function [4, 7].

Applications of EC on the design of cryptographic Boolean functions have focused mainly on functions suitable for use in stream ciphers [24, 25, 47, 48] and masking [25, 47]. Significant effort has also been made to compare the efficiency of various EC approaches, and it has been shown that in nearly all cases GP, CGP, and linear GP significantly outperform other EC methods [24, 47, 48].

4 THE PROPOSED BENCHMARK SUITE

4.1 General Motivation

With our proposed benchmark suite, our primary intention is to promote the synthesis of Boolean functions in GP research since a general benchmark suite for this task is still missing. Therefore, our benchmark suite aims on the synthesis of Boolean functions without any prior knowledge (learning from scratch) to fill a major gap in the benchmarking domain of GP [38]. It is complementary to the benchmark proposed by de Souza et al. [10] which primarily aims on the optimized design of digital circuits. Another general motivation behind our benchmark suite is to promote diversity and accessibility in the Boolean problem domain which we also identified as a gap in the field. More precisely, the work of de Souza et al. does not provide any interfaces and does not include a broad spectrum of Boolean functions. This complicates access and makes it impossible to evaluate the search performance of a GP method from a general perspective. This work was also mainly based on prior experiments [56].

Within the field of GP, a majority of the results in LS were obtained with CGP. The proposal of a general Boolean function benchmark suite can help to increase diversity in LS. More precisely, we think that our proposed benchmark suite can address the following points to create a more diverse and balanced situation within the field of EC:

- Facilitating the reproducibility of existing results
- Establishing a broader spectrum of evaluation results with alternative EC methods
- Gaining more understanding of problem complexity and the corresponding search spaces

Considering the described major points of motivation and identified gaps, we feel that a general, diverse and accessible benchmark suite for Boolean function synthesis is needed.

4.2 Main Objectives and Properties

Considering our general motivation, the philosophy behind our proposed benchmark suite respects the following objectives and properties: **Generalization, Accessibility and Scalability**. A generalized approach to benchmarking in LS is achieved by covering a broader spectrum of types of Boolean functions. However, our approach is balanced with practical orientation since all proposed functions can be implemented in digital circuits. Another property of the benchmark suite is to enable accessibility to the data of the respective benchmarks by providing interfaces for the used formats in three major programming languages: C++, Java, and Python. We make use of the scaling property of Boolean functions by proposing benchmarks of the same type with different bit lengths of the respective inputs. Scaling up the bit length increases the difficulty of the proposed benchmarks but maintains similarity. It has been found that the overuse of single-output functions negatively affects diversity in LS [39, 61]. Moreover, since the overused low-order parity-even and multiplexer benchmarks should be avoided, we only include high-order parity functions and promote diversity by emphasizing multiple-output Boolean functions as it was recommended by White et al. [61].

4.3 Problem Selection

We thoughtfully selected Boolean functions from seven popular categories: arithmetic, transmission, comparison, counting, mixed, parity and cryptography. This categorical selection covers popular subdomains of Boolean functions. In the arithmetic domain, the most popular benchmarks are the adder and multiplier functions which have been commonly used to evaluate the search performance of graph-based GP methods in the past. These types of benchmarks have been comparatively less used in GP history and are currently underrepresented. Besides, the arithmetic benchmarks we selected high dimensional multiple-output functions such as the identity comparator benchmark which remarkably differs from the commonly used single-output parity and multiplexer benchmarks. We also selected the multi-functional arithmetic logic unit as a mixed multi-functional benchmark to expand the scope beyond single-functional benchmarks. The synthesis of cryptographic Boolean functions is a comparatively young and interesting field of application in LS and it is therefore natural to propose these functions as benchmarks.

4.4 Problem Descriptions

Based on a comprehensive survey of relevant papers published in the last two decades, we propose to include the following problems in the LS benchmark suite:

4.4.1 Adders and Multipliers. Adders are a popular arithmetic benchmark that has been used within the EC community since the early days. Adders are naturally targeted as a more viable alternative to the evolution of multipliers. With the original version of CGP, Julian Francis Miller, the pioneer of gate-level evolution, successfully evolved 4-bit adders [41]. The 9+9 bit adder represents the largest problem instance evolved from the randomly initialized seed using a common CGP [23]. Similarly to the multipliers, this adder was evolved on a cluster of computers. The largest instance reported in the literature, however, is the 10+10 adder evolved by means of CGP with a semantically oriented mutation operator [19]. In addition to adders, we also include multipliers. The evolutionary design of multipliers represents probably the hardest problem due to the complexity of the multiplication itself (the multipliers consist of a sequence of adders reducing the partial products to a single output vector) [19]. Various 2×2-bit, 3×3-bit and 4×4-bit multipliers have been successfully evolved in the past not only using computers but also hardware accelerators [41]. Considering this category of benchmarks, the 5×5-bit multiplier represents the most complex instance evolved directly at gate level. Due to the high computational requirements, a single instance of this circuit has been discovered by means of a multi-threaded parallel implementation of CGP running on a supercomputer cluster consisting of 60 computers [23]. Recently, the computational overhead was significantly reduced by the introduction of a semantically oriented mutation operator [19], which helped to reduce the evolution time to less than one hour on a common computer [19]. Nevertheless, the 5×5-bit multiplier is the largest circuit successfully evolved.

4.4.2 One-hot encoder and decoder. Encoders and decoders are another type of multiple-output problem that is used within EC, but less frequently compared to the previous case.

Table 1: Function set of the arithmetic logic unit benchmark

Opcode	Function	Description
000	&	Logical and
001		Logical or
010	⊕	Exclusive or
011	+	Addition
1XX	−	Subtraction

A one-hot encoder circuit is a digital circuit that converts a binary representation of a n -bit number into a one-hot representation. In a one-hot representation that occupies 2^n bits, only one bit is active at a time, and all other bits are low. The active bit indicates the value of the input number, while the low bits are used to represent the other values. The representation is beneficial for implementing state machines and processing categorical data in machine learning. A one-hot encoder can be used in a demultiplexer to generate its selection signal. The demultiplexer benchmark is a multiple-output problem which has been used in former work to evaluate the search performance of graph-based GP systems [1, 28, 29, 58]. The authors reported results for the 3:8-bit and 4:16-bit demultiplexer.

4.4.3 Comparators. We include two types of comparators, the identity and magnitude comparator. Both types check whether one value is less or greater than the other or if both values are equal. The identity comparator is another benchmark which provides a scaleable multiple-output problem that has been proposed by Walker and Miller [58]. The $n \times w$ -bit identity comparator compares all possible pairs of n inputs and determines whether the first value of the pair is less than, equal to, or greater than the second one. This means that for every pair of w -bit values, the comparator provides three binary outputs indicating the comparison result. Results for the 3×1 and 4×1 -bit identity comparator have been reported in former work [1, 28, 29, 58]. In addition to the identity comparator, we also include the magnitude comparator benchmark to our suite. The magnitude comparator compares the sum of two operands whereby the size of each operand depends on the respective bit length.

4.4.4 Ones' counter circuit. A ones' counter circuit is a digital circuit that counts the number of active bits (i.e. ones) in a binary input. The result corresponds to the Hamming weight of the input, which is an important operation applicable in various domains including error correction, data compression, database systems, and machine learning. Due to its frequent use, this operation is present in modern processors in the form of a popcount instruction. The ones' counter can be efficiently implemented using an adder tree circuit which performs parallel addition of adjacent pairs of bits of the binary input until only a single sum remains.

4.4.5 Mixed (multi-functional). Mixed or multi-functional Boolean functions have been comparatively less used in the past. In the field of CGP, experiments have predominantly concentrated on the evaluation of single-functional benchmarks. However, Walker and Miller [58] proposed a 3-bit arithmetic logic unit (ALU) benchmark for the evaluation of a multi-chromosome approach to the CGP representation model. We, therefore, include an ALU benchmark which consists of 3 logical functions and two arithmetic functions that are shown in Table 1.

4.4.6 Parity circuits. Parity circuits are another commonly used benchmark. Although the construction of an optimal parity circuit is a straightforward process, these circuits provide a simple, yet challenging problem that can be used to evaluate and compare different optimization algorithms. Parity circuits are usually designed using a subset of common gates, including AND, OR and NOT gates. But even if we allow the use of XOR gates, the design of parity circuits is difficult due to the presence of a deceptive landscape, especially for higher bit widths [55]. The difficulty comes from the fact that Parity is a symmetric Boolean function where the output only depends on the number of ones. 5,8,10 and 12-input parities have been used to investigate the role of neutrality in CGP [65]. Up to 24-input parity has been evolved using the self modifying CGP approach [14]. Parity circuits were also evolved using CGP with semantically oriented mutation operator and results up to 10-bit were reported [19]

4.4.7 Cryptographic functions. Our benchmark includes four types of cryptographic Boolean functions commonly designed in literature: bent, balanced, resilient, and masking [7, 24, 25, 47, 48]. Each of these function types needs to satisfy a different set of cryptographic properties. Bent functions need to possess maximal nonlinearity. Balanced functions need to be both perfectly balanced and highly nonlinear. Resilient functions need to be perfectly balanced, highly nonlinear, have non-zero correlation immunity and have a high algebraic degree. Finally, masking functions need to have high correlation immunity and minimal but non-zero Hamming weight. The definitions of these properties are provided in Section 4.6.2, and the specific values desired by each function are provided in our repository.

4.5 Analysis of Benchmark Complexity

Boolean function complexity is a fundamental concept in computer science. It refers to the minimum resources required to compute a Boolean function, where the resources can be measured in terms of time, space, or other relevant parameters [60].

In addition to the input-output mapping (i.e. the truth table), we may use so-called normal forms for the representation of Boolean functions. Normal forms describe Boolean functions by Boolean operations arranged in a two-level hierarchy. The disjunctive normal form (DNF) and the conjunctive normal form (CNF) are the two most prominent normal forms. In addition to that, algebraic normal form (ANF) is routinely used in cryptography.

DNF represent Boolean functions as a sum of products of literals (the input variable or its negation), where each product is called a minterm and represents a specific combination of input variables that yields a true output. Both DNF and CNF, which can be understood as a specific type of sum of product, are canonical representations, which implies that each of the representations is unique for a specific Boolean function.

We may use the normal forms for the analysis of the complexity of a Boolean functions but determining the complexity of a Boolean function is in general a challenging problem. We can count, for example, the number of minterms. This metric gives us an estimate of the complexity of the function in terms of the number of input combinations that produce a specific output. Nevertheless, intuitively simple functions may have normal forms of exponential

size. Parity, majority, and other functions involving a variant of counting the active inputs, for example, have this issue despite their intuitive simplicity. Hence, multiple metrics are needed to analyze the complexity of a Boolean function more accurately.

4.5.1 Complexity of common functions. The complexity analysis of the chosen benchmark problems is provided in Table 2. The table is divided into four sections, and the corresponding abbreviations used in it are listed in Table 3.

The first section contains the number of inputs (i.e. input variables) and outputs. This helps to indicate the fitness evaluation complexity because the computational complexity of the Hamming distance calculation determined using a circuit simulator grows exponentially with the number of input combinations.

The second section contains metrics that can be calculated from the truth table. In particular, Hamming weight, symmetry and avalanche effect. Hamming weight counts the number of input combinations that produce a true output. In our case, we provide a normalized Hamming weight (the sum of Hamming weights for each output is divided by the total number of input combinations and the number of outputs). Symmetry gives the percentage of the input variable pairs that can be swapped without affecting the output value. The value 100% (see e.g. parity) means that a Boolean function is symmetric. Symmetric Boolean functions are characterized by the fact that their outputs only depend on the Hamming weights of their inputs. i.e. its output value is the same no matter the order of its arguments. These functions can be represented in a very compact way using ANF. Symmetric functions are the only functions having a known implementation with a number of gates which is linear in the number of input variables and are good candidates in terms of implementation complexity [5]. Finally, the avalanche effect corresponds to the average number of output bits that change if a single input bit is flipped. High symmetry and avalanche effect are desirable properties of cryptographic Boolean functions.

The third section of Table 2 contains the sizes of the normal forms. In addition to DNF and ANF, we included also the size of the benchmark represented using Reduced Ordered Binary Decision Diagrams (ROBDD). The advantage of an ROBDD is that it is canonical for a particular Boolean function and variable order. To build ROBDD, we used the variable order minimizing the ROBDD size. Finally, the size of logic circuits and their depth in multi-level representations is provided in the last section. In particular, we report the results for And-Inverter Graph (AIG) and gate-level representation. AIG used by state-of-the-art EDA tools is a directed acyclic graph (DAG) representation of a Boolean function that consists of AND nodes. The edges of DAG can be inverted to model logic negation. We will discuss features that can make the evolutionary design of certain benchmarks difficult. Parity and Ones counter circuits are two representatives of symmetric Boolean functions. While the Parity has a single output, Ones counter has multiple outputs. The size of normal forms grows rapidly in both cases. The size of the gate-level circuit, however, grows linearly. The complexity for the evolutionary design is rooted in the symmetry property which may create a deceptive landscape [55]. The design of the one-hot encoder becomes more and more challenging as the number of inputs

Table 2: Properties of the selected common functions.

Problem			Truth table metrics			Representation						
						Normal forms & ROBDD			Multi-level			
Identifier	Input count	Output count	Function symmetry	Hamming weight	Avalanche effect	DNF terms	ANF terms	BDD nodes	AIG nodes	AIG depth	MAP gates	MAP depth
add4	9	5	46%	50%	1.8	135	65	43	39	10	44	8
add6	13	7	40%	50%	1.8	607	259	82	61	13	68	10
add8	17	9	36%	50%	1.9	2,519	1,029	133	87	15	97	12
mul3	6	6	42%	29%	1.7	35	27	46	37	11	42	9
mul4	8	8	37%	33%	2.4	145	138	145	83	16	89	13
mul5	10	10	34%	36%	2.9	591	671	440	146	21	157	17
dec4	4	2	0%	69%	0.6	4	8	5	3	2	5	3
dec8	8	3	21%	80%	0.4	12	151	17	12	6	21	7
dec16	16	4	27%	85%	0.2	32	39,062	49	35	14	62	15
enc8	3	8	67%	13%	2.0	8	20	24	12	2	38	8
enc16	4	16	75%	6%	2.0	16	66	64	24	2	109	9
enc32	5	32	65%	3%	2.0	32	212	160	110	9	267	11
icomp5	5	30	40%	33%	8.0	40	50	60	30	2	45	2
icomp7	7	63	29%	33%	12.0	84	105	126	63	2	91	2
icomp9	9	108	22%	33%	16.0	144	180	216	108	2	153	2
mcomp4	8	3	36%	33%	0.6	46	161	33	20	7	40	7
mcomp5	10	3	33%	33%	0.5	94	485	42	25	7	49	9
mcomp6	12	3	32%	33%	0.4	190	1,457	51	31	9	61	9
count4	4	2	100%	56%	1.5	15	10	10	14	7	13	4
count6	6	3	100%	47%	1.8	73	36	28	26	12	33	7
count8	8	3	100%	53%	1.8	293	106	42	46	16	49	9
count10	10	4	100%	45%	1.8	1,228	310	80	76	21	84	12
alu4	11	5	32%	46%	1.4	170	240	81	84	12	125	10
alu6	15	7	29%	47%	1.5	740	1,434	157	132	14	183	12
alu8	19	9	28%	48%	1.6	3,038	10,980	257	184	16	252	14
epar8	8	1	100%	50%	1.0	128	8	8	21	6	14	3
epar9	9	1	100%	50%	1.0	256	9	9	24	8	16	4
epar10	10	1	100%	50%	1.0	512	10	10	27	8	18	4
epar11	11	1	100%	50%	1.0	1,024	11	11	30	8	20	4

Table 3: Abbreviations used for the benchmark functions.

Identifier	Name	Category
add	Adder with carry	Arithmetic
mul	Multiplier	Arithmetic
dec	One-hot decoder	Transmission
enc	One-hot encoder	Transmission
icomp	Identity Comparator	Comparison
mcomp	Magnitude Comparator	Comparison
count	Ones' counter circuit	Counting
alu	Arithmetic Logic Unit	Mixed
epar	Parity Even	Parity
bent	Bent	Cryptography
bal	Balanced	Cryptography
res	Resilient	Cryptography
mask	Masking	Cryptography

increases, because the Hamming weight decreases rapidly with every added input. For example, for a 64-bit encoder, there are only 64 true values out of 4096 values in the truth table. Hence the data used for the fitness calculation are extremely unbalanced. In addition, the gate-level implementation requires gates with very high fan out.

The one-hot decoder is a circuit that can be represented compactly with all representations except the ANF form. The size of the ANF is growing exponentially with the increasing number of outputs. Magnitude and identity comparators both have an unbalanced truth table in favor of a 2:1 ratio of zeros (see Hamming weight column in Table 2). While the magnitude comparator has fixed number of outputs, the number of outputs of the identity comparator grows rapidly with the increasing number of inputs. In addition, a single change in the input combination affects many outputs (see the column Avalanche effect). Hence, the gate-level implementation of the identity comparator thus must contain high fan-out gates to satisfy this behavior. The size of the identity comparator grows linearly with the increasing bit width. Adders can compactly be represented with all representations and their implementation is also compact. The size increases linearly with the increasing number of inputs. This benchmark is thus suitable for evaluation of EA scalability. The Hamming weight is constant, the avalanche effect is very stable. The number of symmetries decreases slightly with the increasing bit width. Unlike adders, multipliers represent a well-known problem in the LS domain that is associated with

exponential complexity. The size of normal forms, as well as the size of ROBDD representing multiplication, grows exponentially with the number of input bits. Due to the nature of multiplication, which involves adding partial products, gate-level implementations are very dense and the gates are tightly connected. As a result, the evolutionary design of multipliers is extremely difficult because there is less design freedom compared to other circuits of the comparable size.

4.5.2 Complexity of cryptographic functions. Because cryptographic Boolean functions are not defined via a specific input-output mapping, their complexity can not be described via the above-defined metrics. Instead, the complexity of their design is determined by their rarity and the lack of gradient information in many cryptographic properties.

Functions with ideal cryptographic properties are rare, and the greater the number of inputs, the lower their relative quantity (e.g., the relative quantity of 8-input bent Boolean functions is approximately $2^{-149.71}$ [53]). The exact quantity of Boolean functions that satisfy a specific combination of multiple criteria is usually unknown (except for functions with seven or fewer inputs), and in some cases, it is unknown if any such functions even exist (e.g., balanced functions with nonlinearity of 118) [47, 48].

The second issue is that many cryptographic properties do not contain useful gradient information. That is, unlike for functions whose quality can be evaluated based on the Hamming distance between their desired and actual outputs, for cryptographic Boolean functions there is often no natural middle step between functions that do and do not possess a certain cryptographic property. This makes it difficult for a fitness function to distinguish the quality of candidate solutions, and guide the heuristic search effectively. Naive fitness formulation can also lead to a fitness landscape with many local optima and disconnected components (i.e., it may be possible for the algorithm to become irrecoverably trapped) [26]. An effective fitness function should therefore also consider the non-cryptographic properties of the function's output or structure, like the values of coefficients in the function's Walsh spectrum [4, 6], or the number of active input variables, both of which have been shown to improve performance [25, 26, 48].

When searching for functions with multiple cryptographic properties, all but one of them are often set as strict requirements for a solution to be considered viable, and the last remaining property is then optimized. For balanced and resilient functions, this last property is usually the nonlinearity, while for masking functions it's the Hamming weight. This approach helps to avoid the issue of not knowing the quantity of functions that satisfy the specific criteria and has led to the discovery of entirely new functions with previously unachievable levels of cryptographic properties [47]. The complexity of the cryptographic benchmarks can therefore be modified in two ways. Either by changing the number of input variables or by changing the desired values of its cryptographic properties. Our selection of the desired values is provided in our repository. The known upper bounds and the best values found so far can both be found in literature [7, 24, 52].

4.6 Evaluation Methods (Fitness Calculation)

4.6.1 Similarity of Boolean Circuits. In the case of Boolean circuit evolution, the Hamming distance is typically used in the fitness function to determine the similarity of a Boolean circuit encoded by a candidate solution to the specification. Typically, the problem specification is provided in the form of a truth table, which defines the output of the circuit for every possible input combination. The Hamming distance then measures the difference between the desired and actual outputs of a circuit. As the distance corresponds to the number of positions in which two Boolean vectors differ, its usage in the fitness function is straightforward as it naturally assigns a lower score to circuits that produce outputs closer to the desired outputs and a higher score to circuits that produce outputs further away from the desired outputs. The circuit with the lowest Hamming distance is considered the best and selected for the next generation.

The actual output values of a candidate circuit required for determining the Hamming distance can be obtained by simulating the encoded circuit using a circuit simulator. In order to efficiently evaluate a large number of candidate circuits during the evolution process, it is crucial for the circuit simulation to be fast. A common solution for gate-level CGP is a bit-level parallel simulation [23, 57]. This approach encodes multiple input vectors into w -bit operands and executes them using bit-wise logic instructions in parallel, resulting in a speedup of w on a w -bit processor (assuming $2^n \geq w$). By utilizing the 64-bit instructions available on off-the-shelf CPUs, the evaluation speed can be increased by a factor of 64. The modern CPUs equipped with AVX instructions offer the possibility to have 256-bit operands. Unfortunately, the behavioral specification in the form of a truth table causes poor scalability because the amount of memory required to store the whole truth table, as well as the number of rows (and hence the number of input combinations that need to be checked against the specification), grows exponentially with the increasing number of inputs. The exponential growth in memory and computation requirements makes the simulation of large circuits infeasible, even with modern computers and such approach is tractable for circuits having less than 20 inputs.

To overcome the limitations of the exhaustive circuit simulation and truth table representation, Vasicek and Sekanina proposed to replace the simulator with Binary Decision Diagrams [56]. In particular, they were interested in a variant known as Reduced Ordered Binary Decision Diagrams (ROBDD) where the variables in the diagram are ordered and appear in a specific order, and each node represents a unique logic function. Similarly to truth tables, they can be used as a canonical form for representing Boolean formulas. The representation is often substantially more compact than traditional normal forms and many Boolean operations can be efficiently executed on top of the ROBDD. On the other hand, the requirement of canonicity makes ROBDDs inefficient in representing certain classes of functions. For example, multipliers are known for their exponential memory requirements for any variable ordering.

The similarity between two circuits, expressed as the Hamming distance, can be performed in a time complexity that is linear to the number of nodes in the ROBDD representing those circuits. The most complex operation is the construction of a ROBDD tree. However, the construction process can be optimized using hashing and

node sharing [56]. This approach assumes that both the specification as well as Boolean circuit encoded by a candidate solution are available as ROBDDs. To achieve that, a synthesis operation which gradually builds ROBDD is called for every gate of both circuits. At the end of this process, corresponding outputs of both circuits are connected using exclusive-or gates. To determine the Hamming distance, we sum up the number of input assignments that evaluate the output of each gate to 1. This operation is supported by BDDs and has linear time complexity.

4.6.2 Evaluation of cryptographic properties. The cryptographic Boolean functions included in our benchmark consider a set of five cryptographic properties: Hamming weight, balancedness, nonlinearity, correlation immunity, and an algebraic degree. For a Boolean function with n inputs these properties are defined as follows [4, 6]: Hamming weight $HW(f)$ is defined as the number of ones in the function's truth table.

Balancedness $BAL(f)$ is defined using the Hamming weight as $BAL(f) = \min(HW(f), 2^n - HW(f))/2^{n-1}$. That is, a function is balanced if, and only if, its truth table contains an equal number of ones and zeroes.

Nonlinearity $NL(f)$ is defined using the function's Walsh spectrum $WS(f)$, which can be obtained via the fast Walsh-Hadamard transformation [4, 6], as $NL(f) = 2^{n-1} - \max(WS(f))/2$. It represents the Hamming distance between the function's truth table and the truth table of the nearest linear function or its complement.

Correlation immunity $CI(f)$ is also defined using the Walsh spectrum. If $WS(f)$ contains no items with $1 \leq HW \leq t$, then $CI(f) = t$. Algebraic degree $DEG(f)$ is defined using the function's algebraic normal form $ANF(f)$, which can be obtained via the fast Möbius transformation [4, 6]. If $ANF(f)$ contains at least one item with $HW \geq d$, then $DEG(f) = d$.

For the purposes of the heuristic search, all cryptographic properties are then normalized into a range $<0, 1>$, where 0 means that the Boolean function completely lacks the property and 1 means that the property has the desired (or even better) value. The fitness function is then defined as the sum of all the required cryptographic properties, and the objective is to maximize its value.

4.7 Interfaces and Resources

The data files for the benchmarks are publicly available in our repository⁵ in several formats. For each benchmark, we provide open-source parameterized Verilog model allowing to easily scale the bit width, synthesized baseline in common PLA and BLIF format as well as the complete input-output mapping in uncompressed and compressed form for ease of use. Due to the limited space, the data formats are described in more details in the repository. Interfaces for C++, Java and Python can be also found in our repository.

5 BASELINE RESULTS

With the intention to enable further comparative studies on logic synthesis in EC, we evaluated the experimental search complexity of the evolutionary design of circuit instances included in our benchmark suite. The baseline results presented in this section were obtained with Cartesian Genetic Programming (CGP).

⁵<https://github.com/boolean-function-benchmarks>

5.1 Cartesian Genetic Programming

In contrast to tree-based GP [33], CGP [40, 42] represents a genetic program via genotype-phenotype mapping as an indexed, acyclic, and directed graph. In this way, CGP can be seen as an extension of the traditional tree-based GP approach. The CGP representation model is based on a rectangular grid or row of nodes. Each genetic program is encoded in the genotype of an individual and is decoded to its corresponding phenotype. A definition of a cartesian genetic program is given in Definition 5.1. Originally, the structure of the graph was represented by a rectangular grid of n_r rows and n_c columns, but later work focused on a representation with one row. The CGP decoding procedure processes groups of genes, and each group refers to a node of the graph, except the last one, which represents the outputs of the phenotype. Each node is represented by two types of genes that index the function number in the GP function set and the node inputs. These nodes are called *function nodes* and execute functions on the input values. The number of input genes depends on the maximum arity n_a of the function set. Table 4 shows the set of all 16 binary Boolean functions. In this complete set, the binary index of each function is equivalent to the vector of its truth table values (i.e. 0=NULL, 1=NOR, 2=INHb, ...). A backward search is conducted to decode the corresponding phenotype. The decoding procedure is done for all output genes. An example of the backward search of the most popular one-row integer representation is illustrated in Figure 1. The backward search starts from the program output and processes all nodes which are linked in the genotype. In this way, only active nodes are processed during evaluation. The genotype in Figure 1 is grouped by its function nodes. The first gene of each group refers to the function number in the corresponding function set. The other genes represent the input connections of the node. Inactive function nodes are shown in gray color and with dashed lines.

Definition 5.1 (Cartesian Genetic Program). A cartesian genetic program is an element of the Cartesian product $\mathcal{N}_i \times \mathcal{N}_f \times \mathcal{N}_o \times \mathcal{F}$:

- $\mathcal{N}_i$ is a finite non-empty set of input nodes
- $\mathcal{N}_f$ is a finite set of function nodes
- $\mathcal{N}_o$ is a finite non-empty set of output nodes
- $\mathcal{F}$ is a finite non-empty set of functions

The number of inputs n_i , outputs n_o , and the length of the genotype are fixed. Every candidate program is represented with $n_r * n_c * (n_a + 1) + n_o$ integers. Even if the length of the genotype is fixed for each candidate program, the length of the corresponding phenotype in CGP is variable, which can be considered as an advantage of the CGP representation. CGP is traditionally used with a $(1 + \lambda)$ evolutionary strategy (ES, Algorithm 1). The $(1 + \lambda)$ -ES often yields multiple individuals with equal fitness. The algorithm promotes these fitness-neutral changes by making sure that if the parent and the offspring are of the same fitness, then the offspring will be selected to become the parent of the next generation. This is based on the idea that the resulting genetic drift will help the algorithm explore new areas of the search space, increasing the efficiency of the algorithm. The new population in each generation consists of the best individual of the previous population and the

Table 4: The set of all 16 binary Boolean functions.

Inputs		Functions																
B	A	NULL	NOR	INHb	NOTa	INHb	NOTa	NOTb	XOR	NAND	AND	XNOR	BUFb	IMPb	BUFa	IMPb	OR	ID
0	0	0	1	0	1	0	1	0	1	0	0	1	0	1	0	1	0	1
1	0	0	0	1	1	0	0	0	1	1	0	0	1	1	0	0	1	1
0	1	0	0	0	0	1	1	1	1	0	0	0	0	0	1	1	1	1
1	1	0	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1

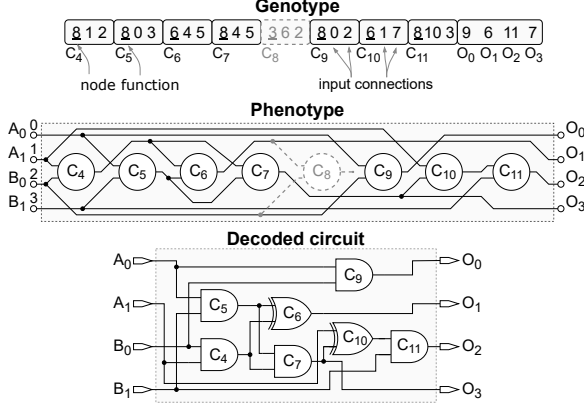


Figure 1: Example of the decoding procedure of a CGP genotype to its corresponding phenotype.

Algorithm 1 $(1+\lambda)$ -ES variant used in CGP

```

1: initialize( $\mathcal{P}$ ) ▷ Initialize parent individual
2: repeat ▷ Until termination criteria not triggered
3:    $Q \leftarrow \text{breed}(\mathcal{P})$  ▷ Breed  $\lambda$  offspring by mutation
4:   Evaluate( $Q$ ) ▷ Evaluate the fitness of the offspring
5:    $Q^+ \leftarrow \text{best}(Q, \mathcal{P})$  ▷ Get individuals which have better fitness as the parent
6:    $Q^- \leftarrow \text{equal}(Q, \mathcal{P})$  ▷ Get individuals which have the same fitness as the parent
7:   ▷ If there exist individuals with better fitness
8:   if  $|Q^+| > 0$  then
9:     ▷ Choose one individual from  $Q^+$  uniformly at random
10:     $\mathcal{P} \leftarrow Q^+[r], r \sim U[0, |Q^+| - 1]$ 
11:   ▷ Otherwise, if there exist individuals with equal fitness
12:   else if  $|Q^-| > 0$  then
13:     ▷ Choose one individual from  $Q^-$  uniformly at random
14:     $\mathcal{P} \leftarrow Q^-[r], r \sim U[0, |Q^-| - 1]$ 
15:   end if
16: until  $\mathcal{P}$  meets termination criterion
17: return  $\mathcal{P}$ 

```

λ created offspring. The breeding procedure is mostly done by a point mutation that swaps genes in the genotype of an individual in the valid range by chance. Another point mutation is the flip of the functional gene, which changes the functional behavior of the corresponding function node [28, 41].

5.2 Function sets

Since the majority of the problems are typically implemented with XOR gates, we propose the use of a reduced function set {BUFa, NOTa, AND, OR} for the easier problems to increase the difficulty

of the evolutionary search. For more complex problems, we recommend an extended function set {BUFa, NOTa, AND, OR, XOR, NAND, NOR, XNOR}. For cryptographic benchmarks set {AND, XOR, OR, XNOR, INHb} should be used [47, 48].

5.3 Experiments

For our experiments, we used standard CGP with the $(1 + \lambda)$ evolutionary strategy that has been described in Algorithm 1 and the standard probabilistic point mutation operator. To report fair results with CGP, we performed hyperparameter optimization (HPO) of the node count, population size and mutation rate with irace [36] with the proposed benchmarks. Since our HPO experiments showed that a setting of $\lambda = 1$ is a general effective choice for our proposed benchmarks we did not list the setting of the λ parameter explicitly. Moreover, this finding is coherent with former results [19, 29]. With our HPO experiments we also identified a mutation rate of 0.3% as a general effective choice for our experiments. For the evaluation of each benchmark, we performed 100 runs and measured the number of fitness evaluations until the CGP algorithm terminated. We defined a limit of 10^8 fitness evaluations for our experiments. Table 5 lists the benchmarks of our suite and presents the corresponding baseline results. Abbreviations used in Table 5 are listed in Table 3. For the 5×5 -bit multiplier benchmark, we only observed unsuccessful runs with CGP.

6 DISCUSSION

There are a few points, that are worthy of discussion and which should be considered by future work. One point which should be addressed is the evaluation method for the search performance of our benchmarks. For the evaluation and presentation of baseline results, we used the number of fitness evaluations until the evolutionary run terminated which can be considered as the predominant method in the field of CGP. Even though this method has been commonly used in the past, this evaluation method has shortcomings of measuring the computational effort which is needed to produce the results by the used GP system. Moreover, this computational effort often depends on the configuration of the respective GP system. For instance, the runtime of an evolutionary run performed by CGP depends on the number of function nodes that are used for each genotype. However, since the benchmarks are typically evaluated on a high-performance computing cluster (HPC) to obtain results in a reasonable time, the runtime of these problems is not accurately measurable due to the noise on HPCs produced by the amount of jobs running in parallel. Regarding achieving a 100% success rate for many instances, we acknowledge that the CGP algorithm has been tested with a relatively large budget of

Table 5: Baseline results obtained in our experiments. Search performance was measured by the number of fitness evaluations (FE) to termination (only successful runs are considered in the statistic).

Problem			Configuration		Search Performance Evaluation					
Identifier	Inputs count	Output count	Function Set	Column count	Average	Standard deviation	Q1	Median	Q3	Success rate
add4	9	5	reduced	5000	1,176,250	1,241,222	559,810	864,099	1,244,870	100%
add6	13	7	reduced	5000	4,354,846	3,425,693	2,098,854	3,358,271	4,883,583	100%
add8	17	9	reduced	5000	15,118,552	12,166,477	6,549,025	11,308,473	18,063,144	100%
mul3	6	6	reduced	4000	439,106	337,382	229,458	317,891	524,669	100%
mul4	8	8	extended	7000	37,074,573	16,213,144	23,682,377	36,943,707	47,053,177	100%
mul5	10	10	extended	7000	-	-	-	-	-	0%
dec4	4	2	extended	1000	11,520	11,199	4,722	8,326	12,079	100%
dec8	8	3	extended	1000	656,502	612,189	277,681	449,462	826,220	100%
dec16	16	4	extended	1000	16,657,577	9,659,375	9,822,953	13,514,673	20,891,128	100%
enc8	3	8	extended	10000	22,895	12,751	13,383	20,015	29,372	100%
enc16	4	16	extended	10000	133,268	55,065	90,761	125,830	163,579	100%
enc32	5	32	extended	10000	6,048,000	5,253,472	1,669,344	4645,978	8,446,886	100%
icomp5	5	30	reduced	3000	298,378	253,280	148,066	231,331	351,607	100%
icomp7	7	63	reduced	3000	1,476,305	771,096	906,376	1,318,098	1,812,708	100%
icomp9	9	108	reduced	3000	5,472,300	2,475,782	3,761,209	5,102,597	6,401,704	100%
mcomp4	8	3	extended	1000	1,418,028	951,804	733,138	1,033,987	2,051,997	100%
mcomp5	10	3	extended	1000	2,898,336	1,923,573	1,593,730	2,489,199	3,416,328	100%
mcomp6	12	3	extended	1000	7,975,125	5,560,237	4,075,013	6,523,368	10,284,085	100%
count4	4	3	extended	1000	44,626	42,380	19,567	30,017	47,265	100%
count6	6	4	extended	1000	853,994	890,100	368,770	649,935	1,108,540	100%
count8	8	4	extended	1000	8,506,276	8,519,358	2,992,012	6,490,877	9,768,313	100%
count10	10	5	extended	1000	29,674,777	17,411,768	16,582,947	26,897,332	40,192,421	86%
alu4	11	5	extended	1000	11,342,677	5,995,435	6,961,372	10,437,724	14,009,597	100%
alu6	15	7	extended	1000	45,341,485	20,465,133	26,869,723	42,071,437	59,961,675	93%
alu8	19	9	extended	1000	72,811,646	16,498,376	63,702,578	74,942,768	85,688,638	23%
epar8	8	1	reduced	1000	2,215,101	1,436,094	1,147,611	1,877,298	2,709,974	100%
epar9	9	1	reduced	1000	4,232,008	2,935,317	2,130,972	3,375,780	5,159,903	100%
epar10	10	1	reduced	1000	8,155,620	6,950,750	3,518,499	5,847,038	10,574,672	100%
epar11	11	1	reduced	1000	11,869,051	9,964,340	6,123,499	8,183,564	13,465,052	100%
bent8	8	1	crypto	500	1,831	4,736	514	881	1,504	100%
bent12	12	1	crypto	500	3,262	2,764	1,543	2,514	4,045	100%
bent16	16	1	crypto	500	8,128	8,161	2,867	5,219	10,067	100%
bal8	8	1	crypto	1000	103,807	124,303	27,635	68,280	143,775	100%
bal12	12	1	crypto	1000	394,556	438,565	101,052	218,533	543,450	100%
res8	8	1	crypto	500	16,712	40,763	3,849	7,062	13,405	100%
res12	12	1	crypto	500	170,153	178,829	39,825	100,651	248,155	100%
mask8	8	1	crypto	1500	3,406	3,694	1,536	2,481	3,919	100%
mask12	12	1	crypto	1500	59,812	60,904	23,446	38,928	75,591	100%

10^8 fitness evaluations, and we have observed variations in average evaluations across different instances. This variation can serve as a baseline for running time analysis. Note that several problems, e.g., mcomp, alu, and par, appear to pose difficulties for the tested algorithm with increasing dimensions, such that the hardness level of our benchmarks can be assured. Also, from the perspective of understanding, we prefer problems that can help us to illustrate and analyze the behavior of algorithms instead of those that are so difficult that all algorithms are trapped. We chose the number of fitness evaluations needed to find the correct solution as our search performance measure because it provides more information

than a simple success rate. Another point that should be addressed is the hardness of the problems. The presented baseline results show, that the hardness of a certain benchmark does not seem to be entirely correlated just with the number of inputs and outputs when evaluated with CGP. Moreover, since our benchmark suite enables a more general perspective on Boolean function synthesis, our suite could also be used for further study and analysis of the search complexity of Boolean functions. The complexity analysis presented in this paper indicates that Boolean functions may be challenging for heuristic methods for very different reasons. Moreover, our analysis also indicates that it may not be possible to use a

single measure to characterize the complexity. Besides, shifting the focus to the evaluation of multiple-output problems as it has been recommended by White et. al [61], a natural next step would be to study the search behavior of EC methods on these problems and its correlation to the respective geometry. Therefore, our benchmark suite also provides a framework for achieving more analytical knowledge about the hardness and search complexity of Boolean functions when synthesized with EC methods.

6.1 Recommendations and Guidelines

Considering the intended and desired further use of our benchmark suite for comparative benchmarking of heuristic methods, we would like to propose some guidelines and recommendations to ensure fair conditions for comparisons of the search performance. In the first place, we recommend to use HPO for each problem [2] which has been proposed in the framework of best benchmarking practices to the EC community. We recommend using the irace package since it is a well-established and widely accepted HPO framework and has been also used in former CGP literature for comparative studies [29, 30]. To perform fair comparisons in terms of statistical significance we recommend to use a nonparametric statistical test (i.e. two-tailed Mann-Whitney U test [37]). In our baseline experiments normally distributed values could not be guaranteed.

7 CONCLUSIONS AND FUTURE WORK

In this work, we proposed an accessible and diverse benchmark suite for the synthesis of Boolean functions, consisting of seven distinct categories that represent problems of varying levels of difficulty. Overall, the benchmarks cover a wide range of Boolean functions in terms of problem hardness and input-output ratio. The application of this suite can support the evaluation of the search performance and robustness of GP methods in evolutionary-driven synthesis of Boolean functions. We also provided an overview of the complexity characteristics of our proposed benchmarks which can be used for further evaluation and analysis on this subject. In the future, we plan to expand the search objectives of our benchmark suite. In this work, we concentrated on function synthesis from scratch and including optimization objectives that have been used in former LS research is a natural next step.

ACKNOWLEDGMENTS

This work was supported by the Czech Science Foundation project 22-02067S.

REFERENCES

- [1] Timothy Atkinson, Detlef Plump, and Susan Stepney. 2018. Evolving Graphs by Graph Programming. In *Genetic Programming - 21st European Conference, EuroGP 2018, Parma, Italy, April 4-6, 2018, Proceedings (Lecture Notes in Computer Science, Vol. 10781)*, Mauro Castelli, Lukás Sekanina, Mengjie Zhang, Stefano Cagnoni, and Pablo García-Sánchez (Eds.). Springer, 35–51. https://doi.org/10.1007/978-3-319-77553-1_3
- [2] Thomas Bartz-Beielstein, Carola Doerr, Jakob Bossek, Sowmya Chandrasekaran, Tome Eftimov, Andreas Fischbach, Pascal Kerschke, Manuel López-Ibáñez, Katherine M. Malan, Jason H. Moore, Boris Naujoks, Patryk Orzechowski, Vanessa Volz, Markus Wagner, and Thomas Weise. 2020. Benchmarking in Optimization: Best Practice and Open Issues. *CoRR abs/2007.03488* (2020). [arXiv:2007.03488](https://arxiv.org/abs/2007.03488)
- [3] Lashon B. Booker, David E. Goldberg, and John H. Holland. 1989. Classifier Systems and Genetic Algorithms. *Artif. Intell.* 40, 1-3 (1989), 235–282. [https://doi.org/10.1016/0004-3702\(89\)90050-7](https://doi.org/10.1016/0004-3702(89)90050-7)
- [4] Ann Braeken. 2006. *Cryptographic properties of Boolean functions and S-boxes*. Ph.D. Dissertation. Katholieke Universiteit Leuven.
- [5] A. Canteaut and M. Videau. 2005. Symmetric Boolean functions. *IEEE Transactions on Information Theory* 51, 8 (2005), 2791–2811. <https://doi.org/10.1109/TIT.2005.851743>
- [6] Claude Carlet. 2010. *Boolean Functions for Cryptography and Error-Correcting Codes*. Cambridge University Press, 257–397. <https://doi.org/10.1017/CBO9780511780448.011>
- [7] Claude Carlet and Xi Chen. 2017. Constructing low-weight d th-order correlation-immune Boolean functions through the Fourier-Hadamard transform. *IEEE Transactions on Information Theory* 64, 4 (2017), 2969–2978.
- [8] Michael Lynn Cramer. 1985. A Representation for the Adaptive Generation of Simple Sequential Programs. In *Proceedings of the 1st International Conference on Genetic Algorithms, Pittsburgh, PA, USA, July 1985*, John J. Grefenstette (Ed.). Lawrence Erlbaum Associates, 183–187.
- [9] Jacob de Nobel, Furong Ye, Diederick Vermetten, Hao Wang, Carola Doerr, and Thomas Bäck. 2021. IOHexperimenter: Benchmarking Platform for Iterative Optimization Heuristics. *CoRR abs/2111.04077* (2021). [arXiv:2111.04077](https://arxiv.org/abs/2111.04077) <https://doi.org/10.1109/2111.04077>
- [10] Lucas Augusto Müller de Souza, José Eduardo Henriques da Silva, Luciano Jerez Chaves, and Heder Soares Bernardino. 2020. A benchmark suite for designing combinational logic circuits via metaheuristics. *Appl. Soft Comput.* 91 (2020), 106246. <https://doi.org/10.1016/j.asoc.2020.106246>
- [11] Richard Forsyth. 1981. BEAGLE A Darwinian Approach to Pattern Recognition. *Kybernetes* 10, 3 (1981), 159–166. <https://doi.org/10.1108/eb005587>
- [12] Gary D. Hachtel and Fabio Somenzi. 1996. *Logic synthesis and verification algorithms*. Kluwer.
- [13] Nikolaus Hansen, Anne Auger, Raymond Ros, Olaf Mersmann, Tea Tusar, and Dimo Brockhoff. 2021. COCO: a platform for comparing continuous optimizers in a black-box setting. *Optim. Methods Softw.* 36, 1 (2021), 114–144. <https://doi.org/10.1080/10556788.2020.1808977>
- [14] S. Harding, J. F. Miller, and W. Banzhaf. 2009. Self modifying Cartesian Genetic Programming: Parity. In *2009 IEEE Congress on Evolutionary Computation*. 285–292. <https://doi.org/10.1109/CEC.2009.4982960>
- [15] Soha Hassoun and Tsutomu Sasao. 2001. *Logic synthesis and verification*. Vol. 654. Springer Science & Business Media.
- [16] Thomas Helmuth and Peter Kelly. 2021. PSB2: the second program synthesis benchmark suite. In *GECCO '21: Genetic and Evolutionary Computation Conference, Lille, France, July 10-14, 2021*, Francisco Chicano and Krzysztof Krawiec (Eds.). ACM, 785–794. <https://doi.org/10.1145/3449639.3459285>
- [17] Thomas Helmuth and Lee Spector. 2015. General Program Synthesis Benchmark Suite. In *Proceedings of the Genetic and Evolutionary Computation Conference, GECCO 2015, Madrid, Spain, July 11-15, 2015*, Sara Silva and Anna Isabel Esparcia-Alcázar (Eds.). ACM, 1039–1046. <https://doi.org/10.1145/2739480.2754769>
- [18] Joseph Hicklin. 1986. *Application of the Genetic Algorithm to Automatic Program Generation*. Master's thesis. University of Idaho.
- [19] David Hodan, Vojtech Mrazek, and Zdenek Vasicek. 2020. Semantically-Oriented Mutation Operator in Cartesian Genetic Programming for Evolutionary Circuit Design. In *Proceedings of the 2020 Genetic and Evolutionary Computation Conference (Cancún, Mexico) (GECCO '20)*, Association for Computing Machinery, New York, NY, USA, 940–948. <https://doi.org/10.1145/3377930.3390188>
- [20] David Hodan, Vojtech Mrazek, and Zdenek Vasicek. 2021. Semantically-oriented mutation operator in cartesian genetic programming for evolutionary circuit design. *Genet. Program. Evolvable Mach.* 22, 4 (2021), 539–572. <https://doi.org/10.1007/s10710-021-09416-6>
- [21] John H. Holland. 1987. Genetic Algorithms and Classifier Systems: Foundations and Future Directions. In *Proceedings of the 2nd International Conference on Genetic Algorithms, Cambridge, MA, USA, July 1987*, John J. Grefenstette (Ed.). Lawrence Erlbaum Associates, 82–89.
- [22] John H. Holland and Judith S. Reitman. 1977. Cognitive systems based on adaptive algorithms. *SIGART Newsl.* 63 (1977), 49. <https://doi.org/10.1145/1045343.1045373>
- [23] Radek Hrbacek and Lukas Sekanina. 2014. Towards Highly Optimized Cartesian Genetic Programming: From Sequential via SIMD and Thread to Massive Parallel Implementation. In *Proceedings of the 2014 Annual Conference on Genetic and Evolutionary Computation (Vancouver, BC, Canada) (GECCO '14)*, Association for Computing Machinery, New York, NY, USA, 1015–1022. <https://doi.org/10.1145/2576768.2598343>
- [24] Jakub Hrusa. 2019. Comparison of genetic programming methods on design of cryptographic boolean functions. In *European Conference on Genetic Programming*. Springer, 228–244.
- [25] Jakub Hrusa and Lukas Sekanina. 2020. Evolving Cryptographic Boolean Functions with Minimal Multiplicative Complexity. In *2020 IEEE Congress on Evolutionary Computation (CEC)*. IEEE, 1–8.
- [26] Domagoj Jakobovic, Stjepan Picik, Marcella SR Martins, and Markus Wagner. 2021. Toward more efficient heuristic construction of Boolean functions. *Applied Soft Computing* 107 (2021), 107327.
- [27] T. Kalganova. 1997. Evolutionary Approach to Design Multiple-valued Combinational Circuits. In *Proceedings of the 4th International conference on Applications*

- of Computer Systems (ACS'97). Szczecin, Poland, 333–339.
- [28] Roman Kalkreuth. 2019. Two New Mutation Techniques for Cartesian Genetic Programming. In *Proceedings of the 11th International Joint Conference on Computational Intelligence, IJCCI 2019, Vienna, Austria, September 17-19, 2019*, Juan Julián Merelo Guervós, Jonathan M. Garibaldi, Alejandro Linares-Barranco, Kurosh Madani, and Kevin Warwick (Eds.). ScitePress, 82–92. <https://doi.org/10.5220/0008070100820092>
 - [29] Roman Kalkreuth. 2022. Towards Phenotypic Duplication and Inversion in Cartesian Genetic Programming. In *Proceedings of the 14th International Joint Conference on Computational Intelligence, IJCCI 2022, Valletta, Malta, October 24-26, 2022*, Thomas Bäck, Bas van Stein, Christian Wagner, Jonathan M. Garibaldi, H. K. Lam, Marie Cottrell, Faiyaz Doctor, Joaquim Filipe, Kevin Warwick, and Janusz Kacprzyk (Eds.). SCITEPRESS, 50–61. <https://doi.org/10.5220/0011551000003332>
 - [30] Paul Kaufmann and Roman Kalkreuth. 2017. Parametrizing Cartesian Genetic Programming: An Empirical Study. In *KI 2017: Advances in Artificial Intelligence - 40th Annual German Conference on AI, Dortmund, Germany, September 25-29, 2017, Proceedings (Lecture Notes in Computer Science, Vol. 10505)*, Gabriele Kern-Isberner, Johannes Fürnkranz, and Matthias Thimm (Eds.). Springer, 316–322. https://doi.org/10.1007/978-3-319-67190-1_26
 - [31] John R. Koza. 1989. Hierarchical Genetic Algorithms Operating on Populations of Computer Programs. In *Proceedings of the 11th International Joint Conference on Artificial Intelligence, Detroit, MI, USA, August 1989*, N. S. Sridharan (Ed.). Morgan Kaufmann, 768–774. <http://ijcai.org/Proceedings/89-1/Papers/123.pdf>
 - [32] John R. Koza. 1990. Concept Formation and Decision Tree Induction Using the Genetic Programming Paradigm. In *Parallel Problem Solving from Nature, 1st Workshop, PPSN I, Dortmund, Germany, October 1-3, 1990, Proceedings (Lecture Notes in Computer Science, Vol. 496)*, Hans-Paul Schwefel and Reinhard Männer (Eds.). Springer, 124–128. <https://doi.org/10.1007/BFb0029742>
 - [33] John R. Koza. 1990. *Genetic Programming: A Paradigm for Genetically Breeding Populations of Computer Programs to Solve Problems*. Technical Report. Stanford, CA, USA.
 - [34] John R. Koza. 1990. A Hierarchical Approach to Learning the Boolean Multiplexer Function. In *Proceedings of the First Workshop on Foundations of Genetic Algorithms, Bloomington Campus, Indiana, USA, July 15-18 1990*, Gregory J. E. Rawlins (Ed.). Morgan Kaufmann, 171–192. <https://doi.org/10.1016/b978-0-08-050684-5.50014-8>
 - [35] John R. Koza. 1993. *Genetic programming - on the programming of computers by means of natural selection*. MIT Press.
 - [36] Manuel López-Ibáñez, Jérémie Dubois-Lacoste, Leslie Pérez Cáceres, Thomas Stützle, and Mauro Birattari. 2016. The irace package: Iterated Racing for Automatic Algorithm Configuration. *Operations Research Perspectives* 3 (2016), 43–58. <https://doi.org/10.1016/j.orp.2016.09.002>
 - [37] H. B. Mann and D. R. Whitney. 1947. On a Test of Whether one of Two Random Variables is Stochastically Larger than the Other. *The Annals of Mathematical Statistics* 18, 1 (1947), 50–60. <https://doi.org/10.1214/aoms/1177730491>
 - [38] James McDermott, Gabriel Kronberger, Patryk Orzechowski, Leonardo Vanneschi, Luca Manzoni, Roman Kalkreuth, and Mauro Castelli. 2022. Genetic Programming Benchmarks: Looking Back and Looking Forward. *SIGEVOLUTION* 15, 3, Article 1 (dec 2022), 19 pages. <https://doi.org/10.1145/3578482.3578483>
 - [39] James McDermott, David Robert White, Sean Luke, Luca Manzoni, Mauro Castelli, Leonardo Vanneschi, Wojciech Jaskowski, Krzysztof Krawiec, Robin Harper, Kenneth A. De Jong, and Una-May O'Reilly. 2012. Genetic programming needs better benchmarks. In *Genetic and Evolutionary Computation Conference, GECCO '12, Philadelphia, PA, USA, July 7-11, 2012*, Terence Soule and Jason H. Moore (Eds.). ACM, 791–798. <https://doi.org/10.1145/2330163.2330273>
 - [40] Julian F. Miller. 1999. An empirical study of the efficiency of learning boolean functions using a Cartesian Genetic Programming approach. In *Proceedings of the Genetic and Evolutionary Computation Conference*, Wolfgang Banzhaf, Jason Daida, Agoston E. Eiben, Max H. Garzon, Vasant Honavar, Mark Jakiela, and Robert E. Smith (Eds.), Vol. 2. Morgan Kaufmann, Orlando, Florida, USA, 1135–1142. <http://citeseer.ist.psu.edu/153431.html>
 - [41] Julian F. Miller. 2011. *Cartesian Genetic Programming*. Springer Berlin Heidelberg, Berlin, Heidelberg, 17–34. https://doi.org/10.1007/978-3-642-17310-3_2
 - [42] Julian F. Miller and Peter Thomson. 2000. Cartesian Genetic Programming. In *Genetic Programming, European Conference, Edinburgh, Scotland, UK, April 15-16, 2000, Proceedings (Lecture Notes in Computer Science, Vol. 1802)*, Riccardo Poli, Wolfgang Banzhaf, William B. Langdon, Julian F. Miller, Peter Nordin, and Terence C. Fogarty (Eds.). Springer, 121–132. https://doi.org/10.1007/978-3-540-46239-2_9
 - [43] J. F. Miller, P. Thomson, and T. Fogarty. 1997. Designing Electronic Circuits Using Evolutionary Algorithms. Arithmetic Circuits: A Case Study. In *Genetic Algorithms and Evolution Strategies in Engineering and Computer Science*. Wiley, 105–131.
 - [44] Aneta Neumann, Frank Neumann, and Chao Qian. 2022. Evolutionary sub-modular optimisation: tutorial. In *GECCO '22: Genetic and Evolutionary Computation Conference, Companion Volume, Boston, Massachusetts, USA, July 9 - 13, 2022*, Jonathan E. Fieldsend and Markus Wagner (Eds.). ACM, 1427–1449. <https://doi.org/10.1145/3520304.3533624>
 - [45] Patryk Orzechowski, William La Cava, and Jason H. Moore. 2018. Where Are We Now? A Large Benchmark Study of Recent Symbolic Regression Methods. In *Proceedings of the Genetic and Evolutionary Computation Conference (Kyoto, Japan) (GECCO '18)*. Association for Computing Machinery, New York, NY, USA, 1183–1190. <https://doi.org/10.1145/3205455.3205539>
 - [46] Patryk Orzechowski and Jason H. Moore. 2021. Generative and reproducible benchmarks for comprehensive evaluation of machine learning classifiers. *CoRR* abs/2107.06475 (2021). [arXiv:2107.06475](https://arxiv.org/abs/2107.06475) <https://arxiv.org/abs/2107.06475>
 - [47] Stjepan Picek, Claude Carlet, Sylvain Guilley, Julian F. Miller, and Domagoj Jakobovic. 2016. Evolutionary algorithms for boolean functions in diverse domains of cryptography. *Evolutionary computation* 24, 4 (2016), 667–694.
 - [48] Stjepan Picek, Domagoj Jakobovic, Julian F. Miller, Lejla Batina, and Marko Cupic. 2016. Cryptographic Boolean functions: One output, many design criteria. *Applied Soft Computing* 40 (2016), 635–653.
 - [49] Riccardo Poli. 1997. Evolution of Graph-Like Programs with Parallel Distributed Genetic Programming. In *Proceedings of the 7th International Conference on Genetic Algorithms, East Lansing, MI, USA, July 19-23, 1997*, Thomas Bäck (Ed.). Morgan Kaufmann, 346–353.
 - [50] Lukás Sekanina. 2012. Evolvable Hardware. In *Handbook of Natural Computing*, Grzegorz Rozenberg, Thomas Bäck, and Joost N. Kok (Eds.). Springer, 1657–1705. https://doi.org/10.1007/978-3-540-92910-9_50
 - [51] Léo François Dal Piccol Sotto, Paul Kaufmann, Timothy Atkinson, Roman Kalkreuth, and Márcio Porto Basgalupp. 2021. Graph representations in genetic programming. *Genet. Program. Evolvable Mach.* 22, 4 (2021), 607–636. <https://doi.org/10.1007/s10710-021-09413-9>
 - [52] Xiaohu Tang, Deng Tang, Xiangyong Zeng, and Lei Hu. 2010. Balanced Boolean Functions with (Almost) Optimal Algebraic Immunity and Very High Nonlinearity. *IACR Cryptology ePrint Archive* 2010 (2010), 443.
 - [53] Natalia Tokareva. 2011. On the number of bent functions from iterative constructions: lower bounds and hypotheses. *Cryptology ePrint Archive* (2011).
 - [54] Konstantinos Varelas, Ouassim Ait ElHara, Dima Brockhoff, Nikolaus Hansen, Duc Manh Nguyen, Tea Tusar, and Anne Auger. 2020. Benchmarking large-scale continuous optimizers: The bbob-largescale testbed, a COCO software guide and beyond. *Appl. Soft Comput.* 97, Part (2020), 106737. <https://doi.org/10.1016/j.asoc.2020.106737>
 - [55] Zdenek Vasicek. 2017. Bridging the Gap Between Evolvable Hardware and Industry Using Cartesian Genetic Programming. In *Inspired by Nature: Essays Presented to Julian F. Miller on the Occasion of his 60th Birthday*, Susan Stepney and Andrew Adamatzky (Eds.). Emergence, Complexity and Computation, Vol. 28. Springer, Chapter 2, 39–55. https://doi.org/doi:10.1007/978-3-319-67997-6_2
 - [56] Zdenek Vasicek and Lukas Sekanina. 2014. How to evolve complex combinational circuits from scratch?. In *2014 IEEE International Conference on Evolvable Systems, ICES 2014, Orlando, FL, USA, December 9-12, 2014*. IEEE, 133–140. <https://doi.org/10.1109/ICES.2014.7008732>
 - [57] Zdenek Vasicek and Karel Slany. 2012. Efficient Phenotype Evaluation in Cartesian Genetic Programming. In *Lecture Notes in Computer Science*. Springer Berlin Heidelberg, 266–278. https://doi.org/10.1007/978-3-642-29139-5_23
 - [58] James Alfred Walker, Katharina Völz, Stephen L. Smith, and Julian Francis Miller. 2009. Parallel evolution using multi-chromosome cartesian genetic programming. *Genet. Program. Evolvable Mach.* 10, 4 (2009), 417–445. <https://doi.org/10.1007/s10710-009-9093-2>
 - [59] Hao Wang, Diederick Vermetten, Furong Ye, Carola Doerr, and Thomas Bäck. 2022. IOHanalyzer: Detailed Performance Analyses for Iterative Optimization Heuristics. *ACM Trans. Evol. Learn. Optim.* 2, 1 (2022), 3:1–3:29. <https://doi.org/10.1145/3510426>
 - [60] Ingo Wegener. 1991. *The Complexity of Boolean Functions* (Wiley Teubner on Applicable Theory in Computer Science). (1991).
 - [61] David Robert White, James McDermott, Mauro Castelli, Luca Manzoni, Brian W. Goldman, Gabriel Kronberger, Wojciech Jaskowski, Una-May O'Reilly, and Sean Luke. 2013. Better GP benchmarks: community survey results and proposals. *Genet. Program. Evolvable Mach.* 14, 1 (2013), 3–29. <https://doi.org/10.1007/s10710-012-9177-2>
 - [62] Stewart W. Wilson. 1987. Classifier Systems and the Animat Problem. *Mach. Learn.* 2, 3 (1987), 199–228. <https://doi.org/10.1007/BF00058679>
 - [63] S. Yang. 1991. *Logic Synthesis and Optimization Benchmarks User Guide: Version 3.0*. Technical Report. MCNC Technical Report.
 - [64] Furong Ye, Hao Wang, Carola Doerr, and Thomas Bäck. 2020. Benchmarking a $(\mu + \lambda)$ Genetic Algorithm with Configurable Crossover Probability. In *Parallel Problem Solving from Nature - PPSN XVI - 16th International Conference, PPSN 2020, Leiden, The Netherlands, September 5-9, 2020, Proceedings, Part II (Lecture Notes in Computer Science, Vol. 12270)*, Thomas Bäck, Mike Preuss, André H. Deutz, Hao Wang, Carola Doerr, Michael T. M. Emmerich, and Heike Trautmann (Eds.). Springer, 699–713. https://doi.org/10.1007/978-3-030-58115-2_49
 - [65] Tina Yu and Julian Miller. 2002. Finding Needles in Haystacks Is Not Hard with Neutrality. In *Genetic Programming, James A. Foster, Evelynne Lutton, Julian Miller, Conor Ryan, and Andrea Tettamanzi (Eds.)*. Springer Berlin Heidelberg, Berlin, Heidelberg, 13–25.